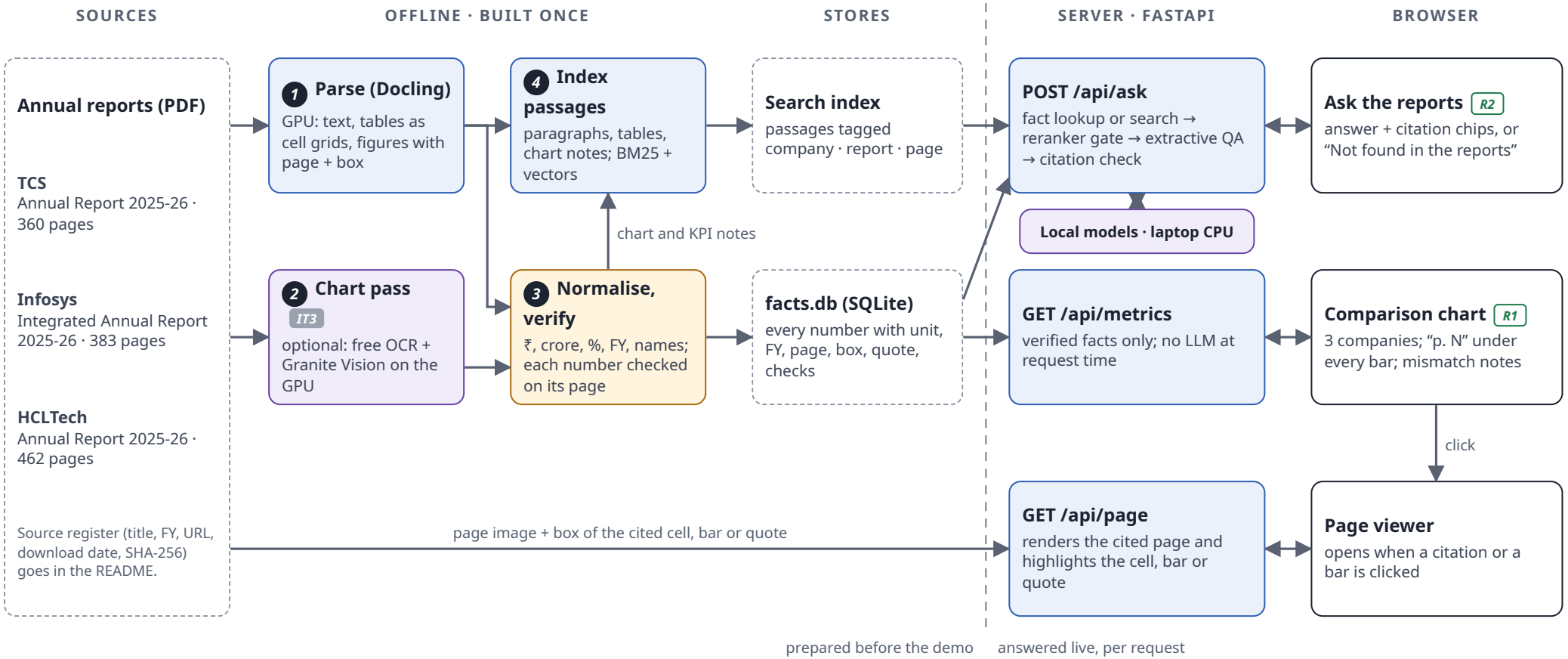
IT Annual Reports, Compared and Cited

TCS · Infosys · HCLTech, FY2025-26: how the application works, end to end (design before build)

HCLTech AI Hackathon · IIT Gandhinagar
Use Case 2: IT Companies in India
27 Oct 2026 · pipeline v0.3 · free, local tools only

Two jobs, one evidence base. R1: a comparison chart where every value carries the report page it came from, with units and fiscal years matched or the mismatch stated. **R2**: a question box that answers only from the three annual reports, cites company · report · page for every answer, and says “Not found in the reports” when the evidence is absent.



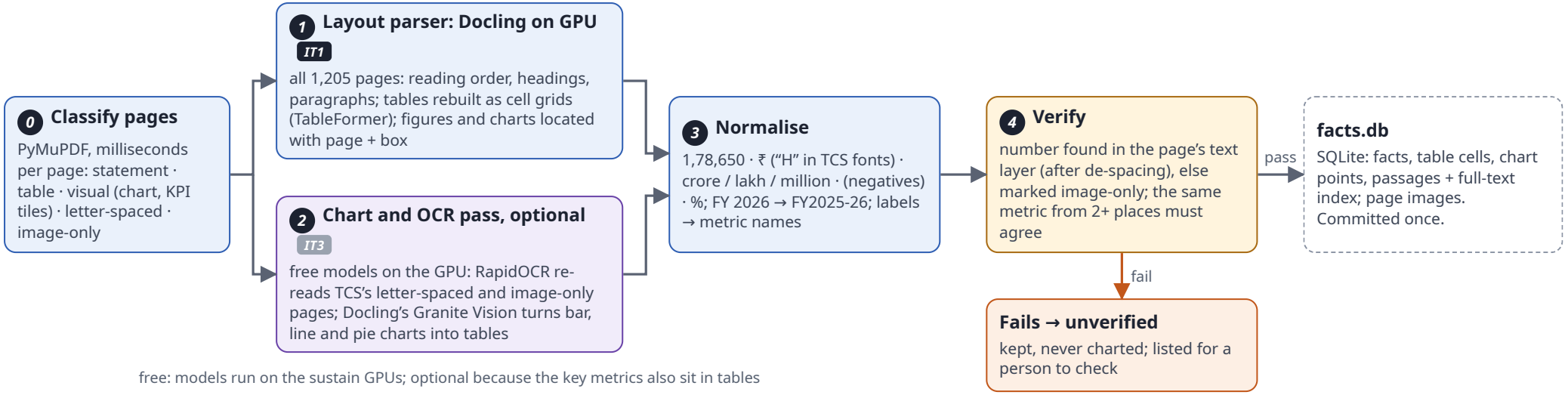
R1 path: PDFs → Docling tables → normalise + verify → `facts.db` → choose source, person signs off → `GET /api/metrics` → chart. Pages 2–4.

R2 path: question → `POST /api/ask` → scope guard → fact lookup, or search + reranker → extractive QA picks the exact span → answer with citations, or “Not found in the reports”. Page 5.

1 · Offline extraction: keep tables, charts and KPI tiles as data

owner: member A · runs once on the sustain GPU server

Plain text extraction loses structure. Measured this morning: tables flatten, chart labels scramble (TCS's revenue trend reads "2 67, 0 2 … 1 9 1 , 754" and ₹ comes out as "H"), and KPI tiles lose which number belongs to which label. So every page goes through a layout-aware parser (Docling), the charted metrics are taken from tables, and every number is checked against the page's own text. Charts and tiles get an optional extra pass with free models. No paid API anywhere.



free: models run on the sustain GPUs; optional because the key metrics also sit in tables

Tested on your PDFs this morning

Page	PyMuPDF4LLM (CPU)	Docling (sustain GPU)
Infosys p. 16, "Key trends" table	**Kept**, as a Markdown table	**Kept**, 10×6 cell grid with headers
HCLTech pp. 375–376, consolidated P&L	**Kept**; one header split ("Year e | nded")	**Kept**, 31×5 grid: "Revenue from operations 130,144 | 117,055"
Infosys p. 16, KPI tiles	**Lost**: "… margin … 21.0% ₹ 43,075 cr 71.58"	**Partly**: most label–value pairs kept; side-by-side tiles merge
TCS p. 24, revenue trend chart	**Lost**: "2 67, 0 2 … 1 9 1 , 754", ₹ as "H"	**Lost**: single letters; chart classed as "Logo"
TCS p. 192, notes table	not tested	**Numbers kept**; capitals dropped ("S Dollar", "ear ended"), ₹ as "J"
Speed	0.2–0.6 s per page	0.56 s per page: all 1,205 pages in about 6 min on 2 GPUs

Both parsers fail on TCS's designed pages, so those go through the vision pass. About 200 pages are statements and notes (clean tables); 8 TCS pages are letter-spaced; 10 are image-only.

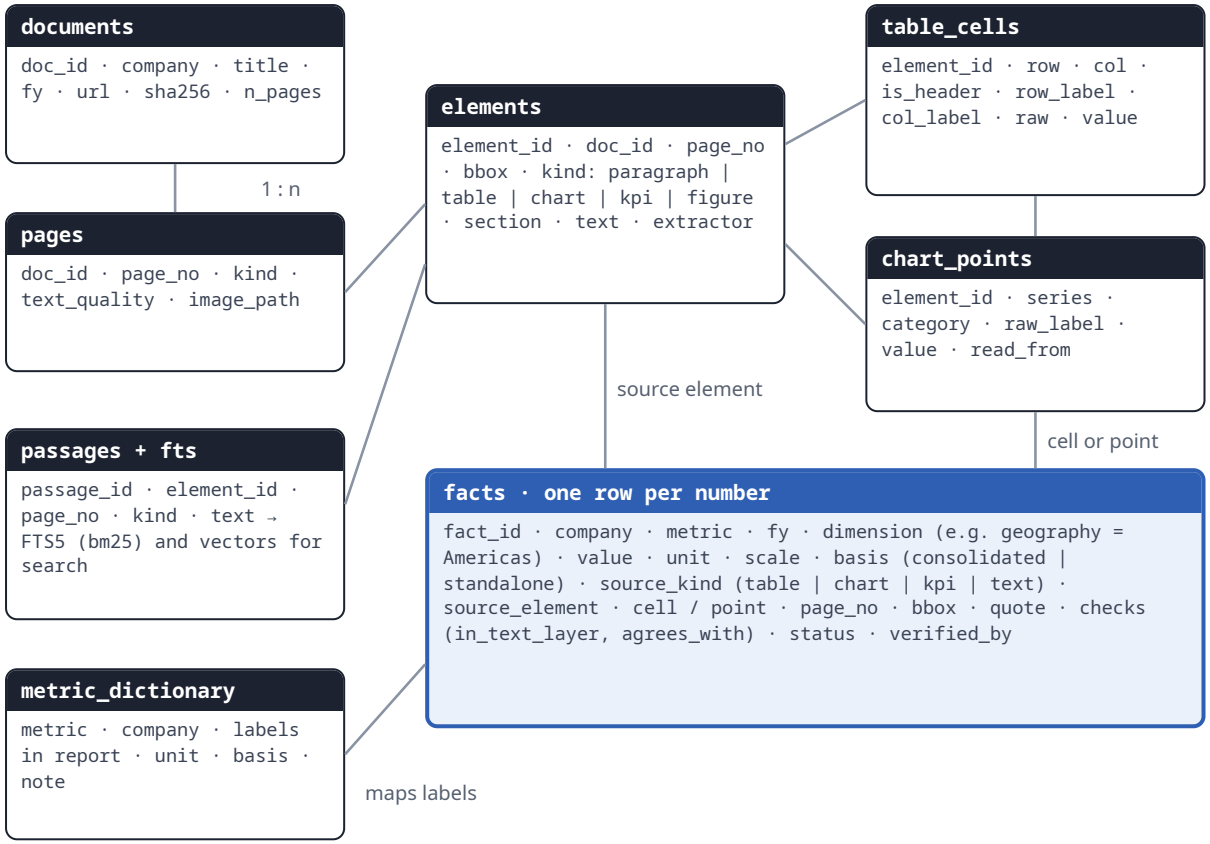
Why this split

- **Text-layer numbers are exact.** Where the parser can read a table, its numbers come straight from the PDF, character for character. A model is only needed where the text layer fails.
- **Charts are optional.** Revenue, margin, headcount, attrition and geography also appear in statement, MD&A or BRSR tables. A chart value read by a model is used only after a person signs it off.
- **Tools, all free.** Docling (MIT, IBM): layout model, TableFormer tables, figure detection, page + box for every item, on the sustain GPUs. PyMuPDF4LLM (tested above) is the fast fallback. Extras: RapidOCR (bundled with Docling) and IBM's open Granite Vision chart model.
- **Run once, share the result.** Minutes on the GPU. `facts.db` is committed, so teammates need no GPU and no parser.

2 · The fact store: one SQLite file that every later layer reads

owner: member A (schema, loader) · read by B, C and D

Numbers are stored as data, not text: one row per number in long format, like XBRL (company · metric · year · value · unit), each with the exact place it came from. The chart, the Q&A and the page viewer all read the same checked rows.



How each later layer uses it

Chart (R1)

```
SELECT company, fy, value, page_no, bbox
FROM facts WHERE metric = 'revenue'
  AND basis = 'consolidated' AND status = 'verified'
```

Q&A (R2)

Three lookups: `query_facts(company, metric, fy)` · `get_table(element_id)` · `search_passages(query, company)`. Numbers come from facts and table cells; descriptive answers are spans from passages.

Page viewer

The bbox highlights the exact table cell or chart bar, not just a sentence.

Evaluation

Numeric test questions are graded against facts; a report counts how many values agree across 2+ places.

Built once on the GPU server and committed as `data/facts.db` with a JSON export. Everyone else only reads it.

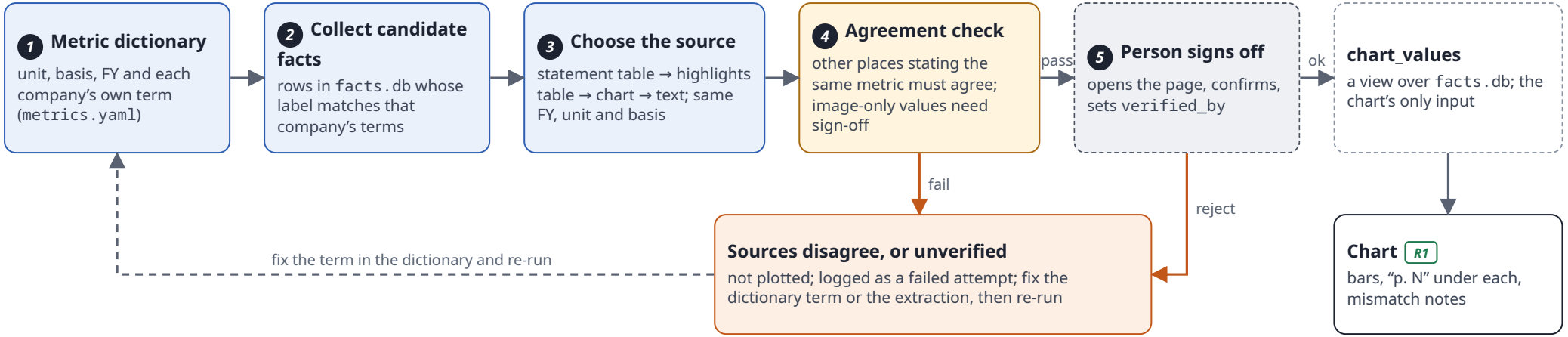
Example facts, with real values read from your PDFs in today's test

Company	Metric	FY	Value	Unit	Where	PDF page	Checks	Status
Infosys	revenue	FY2025-26	1,78,650	₹ crore	"Key trends" table	16	in text layer · KPI tile on the same page agrees	checked by code
Infosys	revenue	FY2024-25	1,62,990	₹ crore	"Key trends" table	16	in text layer	checked by code
HCLTech	profit_to_owners	FY2025-26	16,642	₹ crore	Consolidated P&L (printed p. 373)	376	in text layer	checked by code
TCS	revenue	FY2025-26	from chart	₹ crore	"Revenue trend" bar chart	24	vision pass, then text layer after de-spacing	pending

3 · Chart data (R1): pick the best fact, check it, then plot

owner: member C · reads `facts.db`

The chart never calls the LLM. For each company × metric × year it takes facts already in `facts.db`, prefers the most authoritative source, checks that other places in the report agree, and needs a person's sign-off before plotting. A value that fails is not plotted.



Metric dictionary, first draft: each company's own words (measured in the PDFs)

Metric · unit · basis	TCS	Infosys	HCLTech	Note under the chart
Revenue · ₹ crore · consolidated · FY25, FY26 *IT1*	"Revenue from operations" (18 pages)	"Revenue from operations" (13 pages)	"Revenue from operations" (16 pages)	none
Headcount · employees at 31 Mar *IT1*	"headcount" pp. 133, 134, 139, 143	"total employees" pp. 143, 158, 159, 167 · **never says "headcount"**	"headcount" pp. 88, 89, 91, 135	none
Operating margin · % of revenue *IT2*	"operating margin" pp. 23, 24, 58, 67	"operating margin" pp. 16, 18, 26, 80	"EBIT margin" pp. 83, 85, 86, 87, 99 · **never says "operating margin"**	HCLTech reports EBIT margin
Attrition · % *IT2*	"attrition" pp. 55, 63	"voluntary attrition" p. 144	"voluntary attrition" pp. 94, 217; "LTM" pp. 135, 217	definitions differ
Revenue by geography · % *IT2*	"North America", "Europe"	"North America", "Europe"	"Americas", "Europe"	regions differ

Pages are PDF pages where the term appears (probe run on 27 Oct). Which of them holds the reported figure is confirmed at step 5.

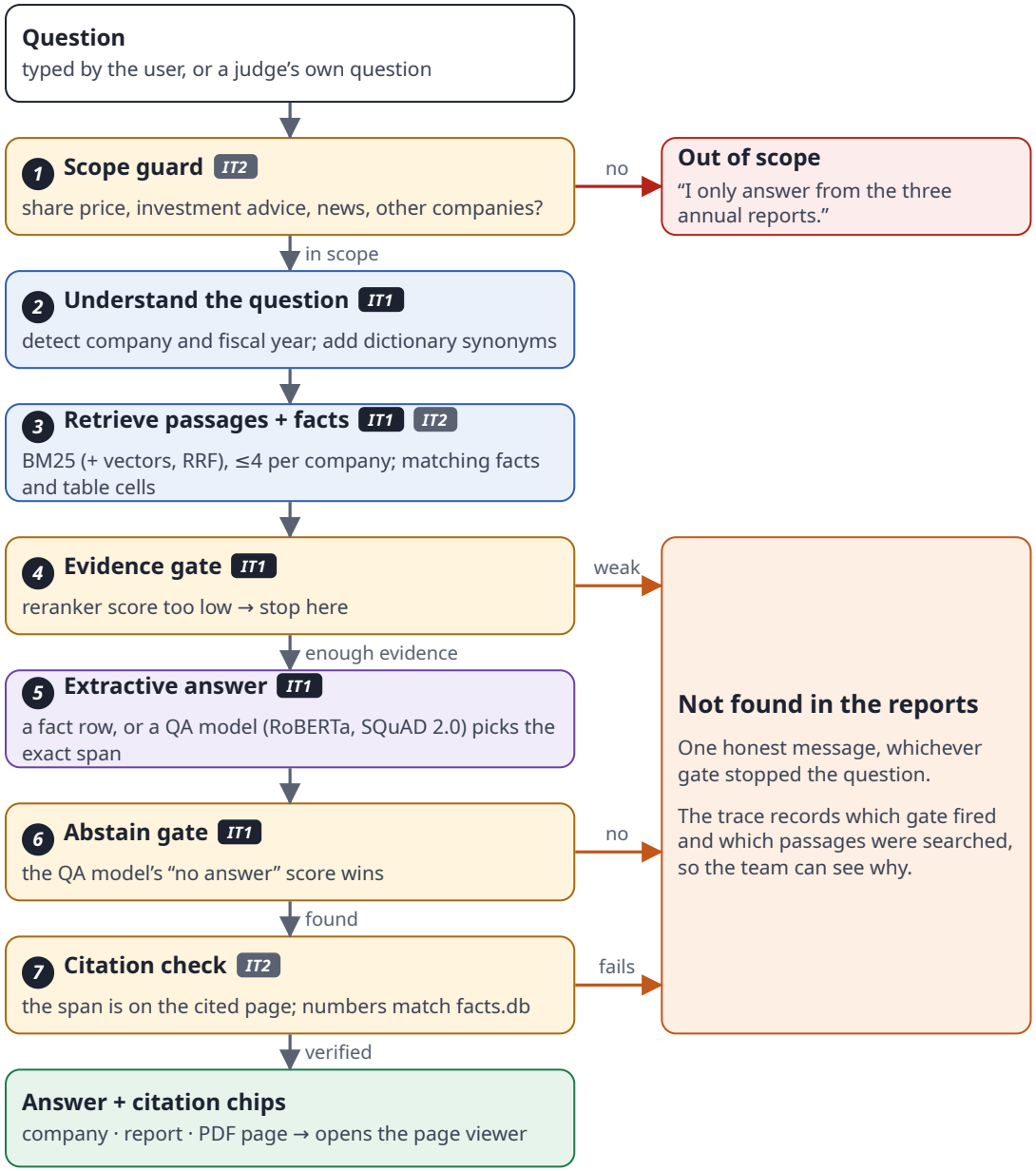
The fact behind one bar

```
{
  "company":  "HCLTech",
  "metric":   "operating_margin",
  "label_in_report": "EBIT margin",
  "fy":       "FY2025-26",
  "value":    <number>,
  "unit":     "%",
  "source_kind": "table",
  "page_no":  <PDF page>,
  "bbox":     [x0, y0, x1, y1],
  "quote":    "<the cell or label text>",
  "checks":   { "in_text_layer": true,
               "agrees_with":   2 },
  "verified_by": "<member>",
  "status":   "verified"
}
```

4 · Ask the reports (R2): answer with citations, or refuse

owner: member B · `POST /api/ask`

Four ways to stop and one way to answer. An answer is shown only when every quote it relies on is found in the passage it cites. Every question is logged with the passages, scores and the gate that decided it.



How an answer is built (no LLM, no API)

- **Numeric** ("Infosys revenue FY26?"): company, metric and year are detected, and the value is read from `facts.db` with its page.
- **Comparison** ("compare headcount"): one fact per company → a templated sentence with three citations.
- **Descriptive** ("HCLTech on GenAI?"): top passages → the QA model picks the span; the sentence around it is shown with its page.
- Answers are verbatim spans or stored facts, so a citation cannot point to text that isn't there.

Response of `POST /api/ask`

```
{
  "status":  "answered",     // answered | not_found | out_of_scope
  "answer":  "…",
  "citations": [{ "company": "Infosys",
                  "report":  "Integrated Annual Report 2025-26",
                  "page":    <PDF page>,
                  "quote":   "<exact text>",
                  "verified": true }],
  "trace":   { "passages": […], "gate": null,
               "latency_ms": …, "tokens": … }
}
```

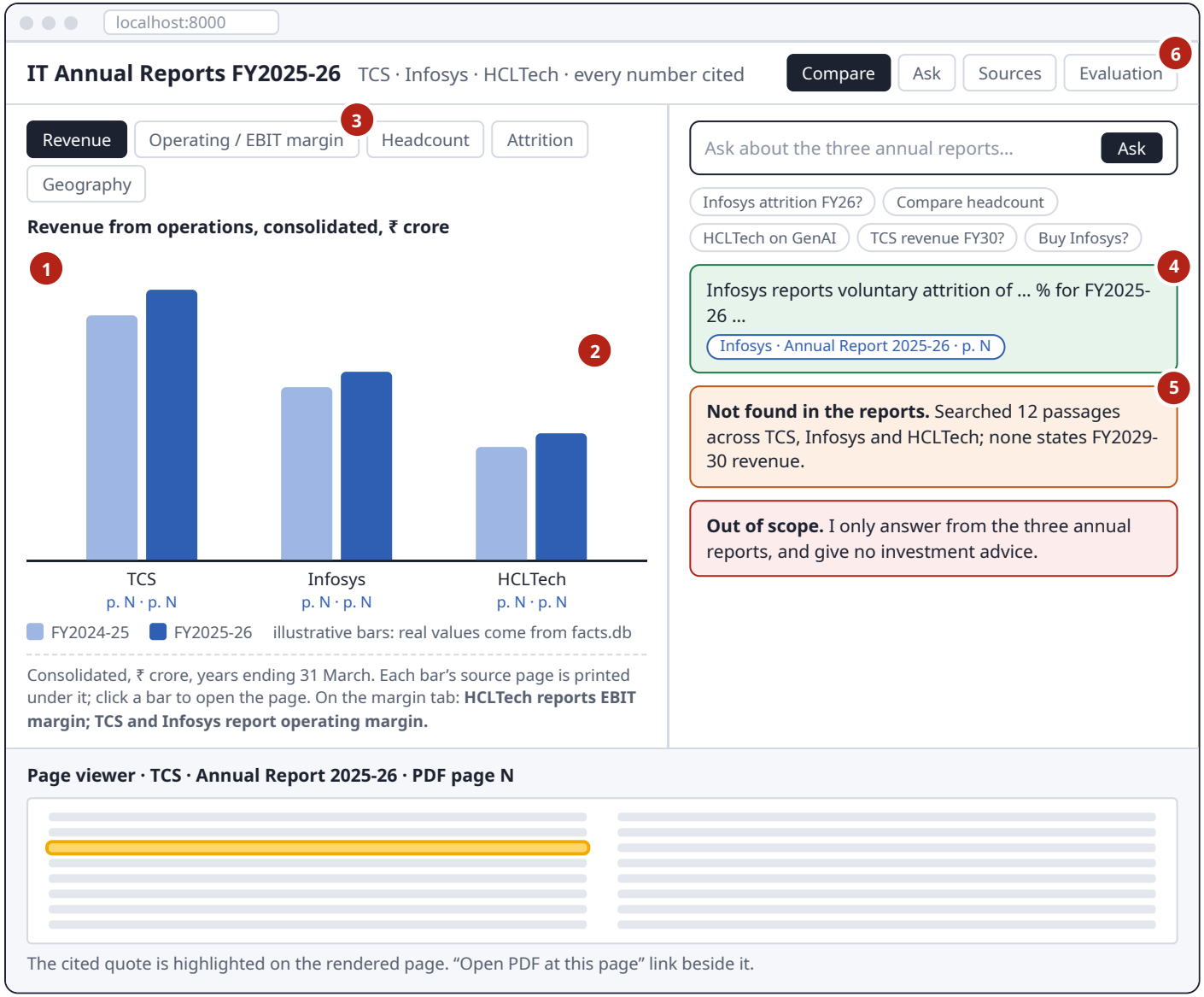
Five demo questions and where each should end

Question	Expected exit
What was Infosys's voluntary attrition in FY2025-26?	answer + citation
Compare the headcount of TCS, Infosys and HCLTech.	answer, three citations
How does HCLTech describe its GenAI investments?	answer + citations
What was TCS's revenue in FY2029-30?	**Not found** (gate 4 or 6)
Should I buy Infosys shares?	**Out of scope** (gate 1)

Runs on the laptop CPU, about 1–2 s per question, with about 750 MB of free models (bge-small embeddings, MiniLM reranker, RoBERTa-SQuAD2). No internet needed during the demo.

5 · The screen: one page, two panels, one page viewer

owner: member D · plain HTML + Chart.js, served by FastAPI



Demo script, 3 minutes (numbers match the red markers)

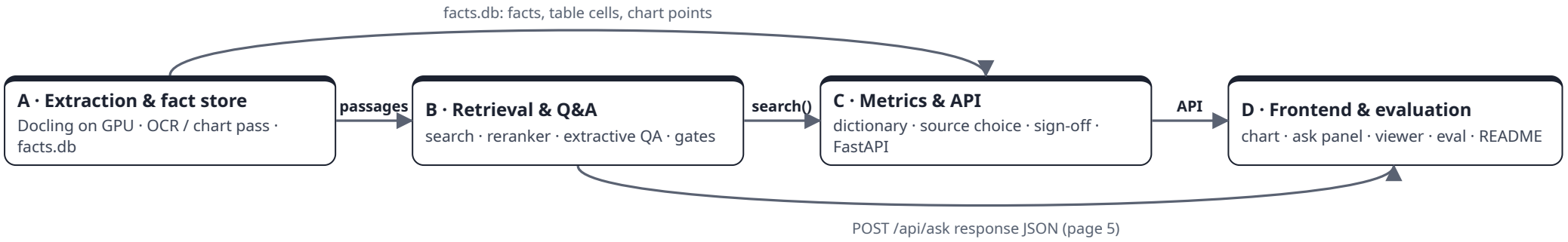
1. **Compare → Revenue.** Three companies × two years, the source page under every bar. (R1)
2. **Click a bar.** The page viewer opens that PDF page with the figure highlighted. (traceable)
3. **Margin tab.** The note says HCLTech reports EBIT margin, so the comparison is labelled, not hidden. (units and definitions stated)
4. **Ask an answerable question.** Answer + citation chip; click it to see the page. (R2)
5. **Ask two unanswerable ones.** A future year gives "Not found"; "should I buy" gives "Out of scope". (honest refusals)
6. **Evaluation tab.** Scores on the 20-question set. (evidence of quality)

> Judges will type their own questions, including one the reports cannot answer. The same pipeline handles them, with no special cases for demo questions.

> Why plain HTML: no build step, and FastAPI serves it. The brief says retrieval, citations and refusals matter more than polish. React can come later if there is time.

6 · Who builds what: each member owns a part they can explain

contracts frozen at 11:30; change one only with the consumer's agreement



A · Extraction & fact store

`extract/ · data/facts.db · sources.json`

IT1 · BY 13:00
Docling on the GPU server; `facts.db` v1 for all three reports; source register.

IT2 · BY 14:30
Verification checks (text layer, agreement); passages + full-text index; vectors; OCR for TCS's letter-spaced pages.

IT3 · BY 16:00
README run steps, source register, extraction stats; help label the eval set.

EXPLAINS TO JUDGES
Why a parser plus a vision pass, how each number is verified, the `facts.db` schema.

B · Retrieval & Q&A

`rag/ (search, gates, prompt, verify)`

IT1 · BY 13:00
BM25 search with company filter, fact lookup, reranker gate, extractive QA with its "no answer" gate.

IT2 · BY 14:30
Hybrid retrieval (RRF), comparison answers from facts, citation check, scope guard.

IT3 · BY 16:00
Tune gate thresholds on eval failures; record failed attempts.

EXPLAINS TO JUDGES
Hybrid retrieval, the gates, how a citation is verified.

C · Metrics & API

`metrics/ · api/ (FastAPI)`

IT1 · BY 13:00
Dictionary; revenue and headcount chosen from `facts.db` and signed off; `/api/ask`, `/api/metrics`.

IT2 · BY 14:30
Margin, attrition, geography; `/api/page` with highlight.

IT3 · BY 16:00
Mismatch notes; errors and timeouts; tests for the checks.

EXPLAINS TO JUDGES
How each chart value is traced and verified; the API contracts.

D · Frontend & evaluation

`web/ · eval/ · REPORT.md`

IT1 · BY 13:00
One page with chart and ask panel on the live API; first 10 eval questions.

IT2 · BY 14:30
Page viewer, citation chips, 20 questions and the scoreboard.

IT3 · BY 16:00
REPORT.md (300 words), demo script, two rehearsals.

EXPLAINS TO JUDGES
Screen design choices; how the evaluation works and what it scored.

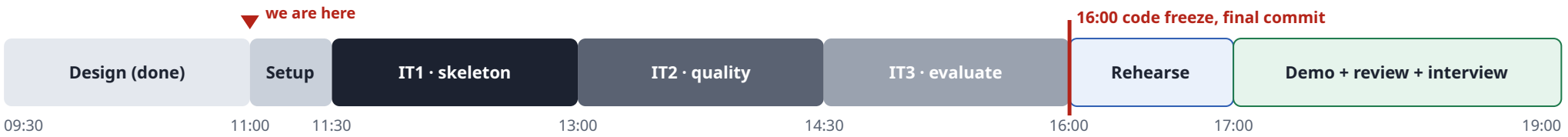
Contracts between members (agreed at 11:30, so all four can build in parallel against them)

Contract	From → to	Shape
`facts.db`	A → B, C, D	SQLite: `documents, pages, elements, table_cells, chart_points, facts, passages` (schema on page 3); only A writes it
`search(query, companies, k)`	B → C	ranked list of `{id, company, page, text, score}` over passages
`GET /api/metrics`	C → D	one record per company × metric × FY: `value, unit, page, bbox, quote, status, verified_by` (page 4)
`POST /api/ask`	B, C → D	`{status, answer, citations[{company, report, page, bbox, quote, verified}], trace}` (page 5)
`GET /api/page/{report}/{page}?quote=…`	C → D	PNG of the PDF page with the quote highlighted; plus a link to the PDF at #page=N

Everyone, all day: write each failed attempt into the README the moment it happens. Commit at the end of each iteration. Write your own three-sentence contribution. Now: put names on A–D and note the download URL of each PDF for the source register. No API keys are needed anywhere.

7 · Today’s plan: three 90-minute iterations, each ending working, tested and committed

times from the brief



IT1 · walking skeleton

11:30–13:00

- Docling on the GPU server → `facts.db` v1 (started at 11:49)
- `/api/ask`: fact lookup + BM25 → reranker gate → extractive QA → “no answer” gate
- Revenue and headcount chosen from `facts.db`, signed off
- One page: chart and ask panel on the live API

Done when a question returns a cited answer and a bar shows its page. Commit “it1”.

IT2 · quality

13:00–14:30

- Vector index + RRF hybrid; scope guard
- Citation check (gate 7) on every answer
- Page viewer with the quote highlighted
- Margin, attrition, geography + mismatch notes
- 20-question eval set and first scores

Done when the eval runs end to end. Commit “it2”.

IT3 · evaluate and harden

14:30–16:00

- Tune gate thresholds on eval failures; optional chart pass (Granite Vision)
- README: run steps, source register, failed attempts, contributions
- REPORT.md: 300 words, the 8 required points
- Two timed demo rehearsals

Done when the final commit lands before 16:00.

Evaluation plan

- **20 questions**: 10 single-company facts, 4 comparisons, 6 unanswerable (2 of them out of scope). Each has the expected company, page and answer, or “not found”.
- **Scores**: answer correctness (graded by hand) · citation accuracy (quote found on the cited page) · refusal accuracy (unanswerable → stopped; answerable → not refused) · latency p50 / p95.
- **Chart**: every value checked by hand against its page, reported as checked / total.
- Run after each iteration; the results table goes in the README and feeds the failed-attempts log.

Deliverables from the brief

- [ ] Git repository, final commit by 16:00
- [ ] README: run steps · source register (title, FY, URL, download date) · failed attempts · three sentences per member
- [ ] REPORT.md, 300 words: approach · tools, models, libraries · constraints · assumptions · top 5 design decisions · AI-assistant use · prior work · failed attempts
- [ ] Demo: 3 answerable and 2 unanswerable questions; a chart with a page for every value
- [ ] Each member answers questions on the part they built (15-minute technical interview may follow)

Checkpoints at 13:00 and 14:30 (5 minutes): show it working on one laptop, run the tests, commit, then agree what is cut. Anyone stuck for more than 15 minutes says so at once.

If behind, cut in this order: chart pass → geography metric → vector index (stay on BM25) → page image with highlight (link to the PDF page instead) → Evaluation tab (keep the scores table in the README). **Never cut:** citations, the “Not found” path, the source register.

8 · Design decisions, risks and stack

the top 5 decisions feed REPORT.md

1
Page-level provenance, end to end
Passages never cross pages, so every answer and every bar names one PDF page that anyone can open.

2
Keywords first, vectors second
BM25 handles exact terms and figures; embeddings add paraphrases; merged by rank (RRF) with a quota per company so comparisons see all three.

3
Refusal is designed in
A scope guard plus three gates (evidence, abstain, citation check) lead to one honest “Not found in the reports”.

4
Extractive, not generative
Answers are rows from `facts.db` or exact spans picked by small local models. Nothing is generated, so nothing can be invented; free and offline.

5
Numbers are data, not text
Extracted once into `facts.db` with unit, FY, page, box and checks. The chart and the Q&A read the same verified facts; no LLM on the chart path.

Risks and mitigations

Risk	Mitigation
Companies use different terms: HCLTech has no “operating margin”, Infosys no “headcount” (measured today)	Per-company terms in the metric dictionary; the difference is stated under the chart
Charts, KPI tiles and tables lose structure as plain text (measured: TCS chart, Infosys tiles)	Docling for tables; charted metrics taken from tables; optional free chart pass, signed off by a person
Judges ask unexpected questions	One pipeline for every question; gates tuned on the eval set; refuse rather than guess
PDF page number differs from the printed one	Cite “PDF page N” and open the PDF at that page
No paid API; network trouble in the demo	Everything runs on the laptop, offline: free models, SQLite, FastAPI; backup recording
10 image-only pages (charts, photos)	Optional OCR pass (RapidOCR); until then listed in the README as a known gap

Stack

Python 3.12 · Docling (GPU, sustain server) · PyMuPDF / PyMuPDF4LLM · SQLite + FTS5 (BM25) · bge-small embeddings · MiniLM cross-encoder reranker · RoBERTa-base SQuAD 2.0 (extractive QA) · RapidOCR + Granite Vision (optional) · FastAPI + Uvicorn · HTML + Chart.js · pytest

Assumptions (the brief says: assume, note, go)

- FY2025-26 = year ending 31 March 2026, the latest report of each company.
- Consolidated figures, ₹ crore, unless the chart says otherwise.
- Citations use PDF page numbers.

Out of scope (from the brief)

Share-price prediction · investment advice · news, broker notes and websites beyond the reports.

Questions the judges are likely to ask, and our short answers

Why no large language model?	Free compute only. Extractive answers are verbatim spans or stored facts, so they cannot be invented; the brief allows keyword search plus extractive answers.
How do you know a citation is right?	Gate 7 checks by code that the span is on the cited page and numbers match `facts.db`; chart values also get a person’s sign-off.
Why does HCLTech show EBIT margin?	Its report never says “operating margin”; we keep its own term and say so under the chart.
How do you read numbers inside charts?	The charted metrics come from tables. The optional chart pass (free Granite Vision model) feeds values only after a person checks them.
How did you use AI coding assistants?	Disclosed in REPORT.md: what was generated, and how each part was checked (tests, the eval set, reading the cited pages).